

CONTINUOUS CONTROL WITH DEEP REINFORCEMENT LEARNING

Timothy P. Lillicrap*, Jonathan J. Hunt*, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver & Daan Wierstra

Google Deepmind

London, UK

{countzero, jjhunt, apritzel, heess, etom, tassa, davidsilver, wierstra} @ google.com

ABSTRACT

We adapt the ideas underlying the success of Deep Q-Learning to the continuous action domain. We present an actor-critic, model-free algorithm based on the deterministic policy gradient that can operate over continuous action spaces. Using the same learning algorithm, network architecture and hyper-parameters, our algorithm robustly solves more than 20 simulated physics tasks, including classic problems such as cartpole swing-up, dexterous manipulation, legged locomotion and car driving. Our algorithm is able to find policies whose performance is competitive with those found by a planning algorithm with full access to the dynamics of the domain and its derivatives. We further demonstrate that for many of the tasks the algorithm can learn policies “end-to-end”: directly from raw pixel inputs.

1 INTRODUCTION

One of the primary goals of the field of artificial intelligence is to solve complex tasks from unprocessed, high-dimensional, sensory input. Recently, significant progress has been made by combining advances in deep learning for sensory processing (Krizhevsky et al., 2012) with reinforcement learning, resulting in the “Deep Q Network” (DQN) algorithm (Mnih et al., 2015) that is capable of human level performance on many Atari video games using unprocessed pixels for input. To do so, deep neural network function approximators were used to estimate the action-value function.

However, while DQN solves problems with high-dimensional observation spaces, it can only handle discrete and low-dimensional action spaces. Many tasks of interest, most notably physical control tasks, have continuous (real valued) and high dimensional action spaces. DQN cannot be straightforwardly applied to continuous domains since it relies on finding the action that maximizes the action-value function, which in the continuous valued case requires an iterative optimization process at every step.

An obvious approach to adapting deep reinforcement learning methods such as DQN to continuous domains is to simply discretize the action space. However, this has many limitations, most notably the curse of dimensionality: the number of actions increases exponentially with the number of degrees of freedom. For example, a 7 degree of freedom system (as in the human arm) with the coarsest discretization $a_i \in \{-k, 0, k\}$ for each joint leads to an action space with dimensionality: $3^7 = 2187$. The situation is even worse for tasks that require fine control of actions as they require a correspondingly finer grained discretization, leading to an explosion of the number of discrete actions. Such large action spaces are difficult to explore efficiently, and thus successfully training DQN-like networks in this context is likely intractable. Additionally, naive discretization of action spaces needlessly throws away information about the structure of the action domain, which may be essential for solving many problems.

In this work we present a model-free, off-policy actor-critic algorithm using deep function approximators that can learn policies in high-dimensional, continuous action spaces. Our work is based

*These authors contributed equally.

通过深度强化学习进行持续控制

Timothy P. Lillicrap*, Jonathan J. Hunt*, Alexander Pritzel, Nicolas Hees, Tom Erez, Yuval Tassa, David Silver 和 Daan Wierstra
Google Deepmind 英国伦敦 {countzero, jjhunt, apritzel, heess, etom, tassa, davidsilver, wierstra} @ google.com

抽象的

我们将深度 Q 学习成功背后的思想应用到连续行动领域。我们提出了一种基于确定性策略梯度的演员批评家、无模型算法，可以在连续动作空间上运行。使用相同的学习算法、网络架构和超参数，我们的算法稳健地解决了 20 多个模拟物理任务，包括推杆摆动、灵巧操纵、腿式运动和汽车驾驶等经典问题。我们的算法能够找到性能与完全访问域及其衍生动态的规划算法所找到的策略具有竞争力的策略。我们进一步证明，对于许多任务，算法可以“端到端”学习策略：直接从原始像素输入中学习。

1 简介

人工智能领域的主要目标之一是通过未经处理的高维感官输入来解决复杂的任务。最近，通过将感官处理深度学习的进步（Krizhevsky 等，2012）与强化学习相结合，取得了重大进展，产生了“深度 Q 网络”（DQN）算法（Mnih 等，2015），该算法能够在使用未经处理的像素作为输入的许多 Atari 视频游戏中达到人类水平的性能。为此，使用深度神经网络函数逼近器来估计动作价值函数。

然而，虽然 DQN 解决了高维观察空间的问题，但它只能处理离散和低维的动作空间。许多感兴趣的任務，尤其是物理控制任务，都具有连续（实值）和高维的动作空间。DQN 不能直接应用于连续域，因为它依赖于寻找最大化动作价值函数的动作，在连续值的情况下，每一步都需要迭代优化过程。

将 DQN 等深度强化学习方法应用于连续域的一个明显方法是简单地离散化动作空间。然而，这有很多限制，最明显的是维数灾难：动作的数量随着自由度的数量呈指数增长。例如，一个 7 自由度系统（如人臂），每个关节的最粗离散化 $a_i \in \{-k, 0, k\}$ 会导致一个维度为 $3^7 = 2187$ 的动作空间。对于需要精细控制动作的任务来说，情况更糟，因为它们需要相应更细粒度的离散化，从而导致离散动作数量的爆炸式增长。如此大的动作空间很难有效地探索，因此在这种情况下成功训练类似 DQN 的网络可能很棘手。此外，动作空间的朴素离散化不必要地丢弃了有关动作域结构的信息，而这对于解决许多问题可能至关重要。

在这项工作中，我们提出了一种使用深度函数逼近器的无模型、离策略演员批评算法，可以在高维、连续动作空间中学习策略。我们的工作是基于

*These authors contributed equally.

on the deterministic policy gradient (DPG) algorithm (Silver et al., 2014) (itself similar to NFQCA (Hafner & Riedmiller, 2011), and similar ideas can be found in (Prokhorov et al., 1997)). However, as we show below, a naive application of this actor-critic method with neural function approximators is unstable for challenging problems.

Here we combine the actor-critic approach with insights from the recent success of Deep Q Network (DQN) (Mnih et al., 2013; 2015). Prior to DQN, it was generally believed that learning value functions using large, non-linear function approximators was difficult and unstable. DQN is able to learn value functions using such function approximators in a stable and robust way due to two innovations: 1. the network is trained off-policy with samples from a replay buffer to minimize correlations between samples; 2. the network is trained with a target Q network to give consistent targets during temporal difference backups. In this work we make use of the same ideas, along with batch normalization (Ioffe & Szegedy, 2015), a recent advance in deep learning.

In order to evaluate our method we constructed a variety of challenging physical control problems that involve complex multi-joint movements, unstable and rich contact dynamics, and gait behavior. Among these are classic problems such as the cartpole swing-up problem, as well as many new domains. A long-standing challenge of robotic control is to learn an action policy directly from raw sensory input such as video. Accordingly, we place a fixed viewpoint camera in the simulator and attempted all tasks using both low-dimensional observations (e.g. joint angles) and directly from pixels.

Our model-free approach which we call Deep DPG (DDPG) can learn competitive policies for all of our tasks using low-dimensional observations (e.g. cartesian coordinates or joint angles) using the same hyper-parameters and network structure. In many cases, we are also able to learn good policies directly from pixels, again keeping hyperparameters and network structure constant¹.

A key feature of the approach is its simplicity: it requires only a straightforward actor-critic architecture and learning algorithm with very few “moving parts”, making it easy to implement and scale to more difficult problems and larger networks. For the physical control problems we compare our results to a baseline computed by a planner (Tassa et al., 2012) that has full access to the underlying simulated dynamics and its derivatives (see supplementary information). Interestingly, DDPG can sometimes find policies that exceed the performance of the planner, in some cases even when learning from pixels (the planner always plans over the underlying low-dimensional state space).

2 BACKGROUND

We consider a standard reinforcement learning setup consisting of an agent interacting with an environment E in discrete timesteps. At each timestep t the agent receives an observation x_t , takes an action a_t and receives a scalar reward r_t . In all the environments considered here the actions are real-valued $a_t \in \mathbb{R}^N$. In general, the environment may be partially observed so that the entire history of the observation, action pairs $s_t = (x_1, a_1, \dots, a_{t-1}, x_t)$ may be required to describe the state. Here, we assumed the environment is fully-observed so $s_t = x_t$.

An agent’s behavior is defined by a policy, π , which maps states to a probability distribution over the actions $\pi: \mathcal{S} \rightarrow \mathcal{P}(\mathcal{A})$. The environment, E , may also be stochastic. We model it as a Markov decision process with a state space $\mathcal{S}$, action space $\mathcal{A} = \mathbb{R}^N$, an initial state distribution $p(s_1)$, transition dynamics $p(s_{t+1}|s_t, a_t)$, and reward function $r(s_t, a_t)$.

The return from a state is defined as the sum of discounted future reward $R_t = \sum_{i=t}^T \gamma^{(i-t)} r(s_i, a_i)$ with a discounting factor $\gamma \in [0, 1]$. Note that the return depends on the actions chosen, and therefore on the policy π , and may be stochastic. The goal in reinforcement learning is to learn a policy which maximizes the expected return from the start distribution $J = \mathbb{E}_{r_i, s_i \sim E, a_i \sim \pi} [R_1]$. We denote the discounted state visitation distribution for a policy π as ρ^π .

The action-value function is used in many reinforcement learning algorithms. It describes the expected return after taking an action a_t in state s_t and thereafter following policy π :

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_i \geq t, s_i > t \sim E, a_i > t \sim \pi} [R_t | s_t, a_t] \quad (1)$$

¹You can view a movie of some of the learned policies at <https://goo.gl/J4PIAz>

关于确定性策略梯度 (DPG) 算法 (Silver et al., 2014) (本身类似于 NFQCA (Hafner & Riedmiller, 2011)), 类似的想法可以在 (Prokhorov et al., 1997) 中找到。然而, 如下所示, 这种带有神经函数逼近器的行动者批评家方法的天真应用对于具有挑战性的问题来说是不稳定的。

在这里, 我们将 actor-critic 方法与 Deep Q Network (DQN) 最近成功的见解相结合 (Mnih 等人, 2013 年; 2015 年)。在 DQN 出现之前, 人们普遍认为使用大型非线性函数逼近器学习价值函数是困难且不稳定的。由于两项创新, DQN 能够使用此类函数逼近器以稳定且鲁棒的方式学习价值函数: 1. 使用重放缓冲区中的样本对网络进行离策略训练, 以最大程度地减少样本之间的相关性; 2. 使用目标 Q 网络对网络进行训练, 以在时间差异备份期间提供一致的目标。在这项工作中, 我们利用了相同的想法以及批量归一化 (Ioffe & Szegedy, 2015), 这是深度学习的最新进展。

为了评估我们的方法, 我们构建了各种具有挑战性的物理控制问题, 其中涉及复杂的多关节运动、不稳定和丰富的接触动力学以及步态行为。其中包括推杆摆动问题等经典问题, 以及许多新领域。机器人控制的一个长期挑战是直接从视频等原始感官输入中学习动作策略。因此, 我们在模拟器中放置一个固定视点相机, 并使用低维观察 (例如关节角度) 和直接从像素尝试所有任务。

我们称为 Deep DPG (DDPG) 的无模型方法可以使用相同的超参数和网络结构, 使用低维观察 (例如笛卡尔坐标或关节角度) 来学习所有任务的竞争策略。在许多情况下, 我们还能直接学习好的策略, 再次保持超参数和网络结构恒定¹。

该方法的一个关键特点是它的简单性: 它只需要一个简单的行动者批评家架构和学习算法, 几乎没有“移动部件”, 使其易于实现和扩展到更困难的问题和更大的网络。对于物理控制问题, 我们将结果与规划器 (Tassa et al., 2012) 计算的基线进行比较, 该规划器可以完全访问底层模拟动力学及其导数 (请参阅补充信息)。有趣的是, DDPG 有时可以找到超出规划器性能的策略, 在某些情况下甚至在从像素学习时也是如此 (规划器总是在底层的低维状态空间上进行规划)。

2 背景

我们考虑一个标准的强化学习设置, 其中包含一个以离散时间步长与环境 E 交互的代理。在每个时间步 t , 代理接收观察值 x_t , 采取操作 a_t 并接收标量奖励 r_t 。在此处考虑的所有环境中, 操作都是实值 $a_t \in \mathbb{R}^N$ 。一般来说, 环境可以被部分观察, 因此可能需要观察的整个历史、动作对 $s_t = (x_1, a_1, \dots, a_{t-1}, x_t)$ 来描述状态。在这里, 我们假设环境是完全观察到的, 因此 $s_t = x_t$ 。

代理的行为由策略 π 定义, 该策略将状态映射到动作 $\pi: \mathcal{S} \rightarrow \mathcal{P}(\mathcal{A})$ 上的概率分布。环境 E 也可能是随机的。我们将其建模为具有状态空间 $\mathcal{S}$ 、动作空间 $\mathcal{A} = \mathbb{R}^N$ 、初始状态分布 $p(s_1)$ 、转移动态 $p(s_{t+1}|s_t, a_t)$ 和奖励函数 $r(s_t, a_t)$ 的马尔可夫决策过程。

状态的回报被定义为折扣未来奖励 $R_t = \sum_{i=t}^T \gamma^{(i-t)} r(s_i, a_i)$ 与折扣因子 $\gamma \in [0, 1]$ 的总和。请注意, 回报取决于所选择的动作, 因此也取决于策略 π , 并且可能是随机的。强化学习的目标是学习一种策略, 使起始分布 $J = \mathbb{E}_{r_i, s_i \sim E, a_i \sim \pi} [R_1]$ 的预期回报最大化。我们将策略 π 的折扣状态访问分布表示为 ρ^π 。

动作值函数被用在许多强化学习算法中。它描述了在状态 s_t 下采取操作 a_t 并随后遵循策略 π 后的预期回报:

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_i \geq t, s_i \geq t \sim E, a_i \geq t \sim \pi} [R_t | s_t, a_t] \quad (1)$$

¹你可以v 在 <https://goo.gl/J> 上观看一些所学政策的电影

Many approaches in reinforcement learning make use of the recursive relationship known as the Bellman equation:

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi} [Q^\pi(s_{t+1}, a_{t+1})]] \quad (2)$$

If the target policy is deterministic we can describe it as a function $\mu : \mathcal{S} \leftarrow \mathcal{A}$ and avoid the inner expectation:

$$Q^\mu(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma Q^\mu(s_{t+1}, \mu(s_{t+1}))] \quad (3)$$

The expectation depends only on the environment. This means that it is possible to learn Q^μ off-policy, using transitions which are generated from a different stochastic behavior policy β .

Q-learning (Watkins & Dayan, 1992), a commonly used off-policy algorithm, uses the greedy policy $\mu(s) = \arg \max_a Q(s, a)$. We consider function approximators parameterized by θ^Q , which we optimize by minimizing the loss:

$$L(\theta^Q) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta, r_t \sim E} [(Q(s_t, a_t | \theta^Q) - y_t)^2] \quad (4)$$

where

$$y_t = r(s_t, a_t) + \gamma Q(s_{t+1}, \mu(s_{t+1}) | \theta^Q). \quad (5)$$

While y_t is also dependent on θ^Q , this is typically ignored.

The use of large, non-linear function approximators for learning value or action-value functions has often been avoided in the past since theoretical performance guarantees are impossible, and practically learning tends to be unstable. Recently, (Mnih et al., 2013; 2015) adapted the Q-learning algorithm in order to make effective use of large neural networks as function approximators. Their algorithm was able to learn to play Atari games from pixels. In order to scale Q-learning they introduced two major changes: the use of a *replay buffer*, and a separate *target network* for calculating y_t . We employ these in the context of DDPG and explain their implementation in the next section.

3 ALGORITHM

It is not possible to straightforwardly apply Q-learning to continuous action spaces, because in continuous spaces finding the greedy policy requires an optimization of a_t at every timestep; this optimization is too slow to be practical with large, unconstrained function approximators and nontrivial action spaces. Instead, here we used an actor-critic approach based on the DPG algorithm (Silver et al., 2014).

The DPG algorithm maintains a parameterized actor function $\mu(s | \theta^\mu)$ which specifies the current policy by deterministically mapping states to a specific action. The critic $Q(s, a)$ is learned using the Bellman equation as in Q-learning. The actor is updated by following the applying the chain rule to the expected return from the start distribution J with respect to the actor parameters:

$$\begin{aligned} \nabla_{\theta^\mu} J &\approx \mathbb{E}_{s_t \sim \rho^\beta} [\nabla_{\theta^\mu} Q(s, a | \theta^Q) |_{s=s_t, a=\mu(s_t | \theta^\mu)}] \\ &= \mathbb{E}_{s_t \sim \rho^\beta} [\nabla_a Q(s, a | \theta^Q) |_{s=s_t, a=\mu(s_t)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) |_{s=s_t}] \end{aligned} \quad (6)$$

Silver et al. (2014) proved that this is the *policy gradient*, the gradient of the policy’s performance².

As with Q learning, introducing non-linear function approximators means that convergence is no longer guaranteed. However, such approximators appear essential in order to learn and generalize on large state spaces. NFQCA (Hafner & Riedmiller, 2011), which uses the same update rules as DPG but with neural network function approximators, uses batch learning for stability, which is intractable for large networks. A minibatch version of NFQCA which does not reset the policy at each update, as would be required to scale to large networks, is equivalent to the original DPG, which we compare to here. Our contribution here is to provide modifications to DPG, inspired by the success of DQN, which allow it to use neural network function approximators to learn in large state and action spaces online. We refer to our algorithm as Deep DPG (DDPG, Algorithm 1).

²In practice, as in commonly done in policy gradient implementations, we ignored the discount in the state-visitation distribution ρ^β .

强化学习中的许多方法都利用称为贝尔曼方程的递归关系：

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi} [Q^\pi(s_{t+1}, a_{t+1})]] \quad (2)$$

如果目标策略是确定性的，我们可以将其描述为函数 $\mu: \mathcal{S} \leftarrow \mathcal{A}$ 并避免内部期望：

$$Q^\mu(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma Q^\mu(s_{t+1}, \mu(s_{t+1}))] \quad (3)$$

期望仅取决于环境。这意味着可以使用从不同随机行为策略 β 生成的转换来学习 Q^μ 非策略。

Q-learning (Watkins & Dayan, 1992) 是一种常用的离策略算法，使用贪婪策略 $\mu(s) = \arg \max_a Q(s, a)$ 。我们考虑由 θ^Q 参数化的函数逼近器，我们通过最小化损失来优化它：

$$L(\theta^Q) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta, r_t \sim E} [(Q(s_t, a_t | \theta^Q) - y_t)^2] \quad (4)$$

在哪里

$$y_t = r(s_t, a_t) + \gamma Q(s_{t+1}, \mu(s_{t+1}) | \theta^Q). \quad (5)$$

虽然 y_t 也依赖于 θ^Q ，但这通常会被忽略。

过去常常避免使用大型非线性函数逼近器来学习值或动作值函数，因为理论上的性能保证是不可能的，并且实际上学习往往不稳定。最近，(Mnih 等人, 2013; 2015) 采用了 Q 学习算法，以便有效地利用大型神经网络作为函数逼近器。他们的算法能够从像素中学习玩 Atari 游戏。为了扩展 Q-learning，他们引入了两个主要变化：使用 *replay buffer* 和单独的 *target network* 来计算 y_t 。我们在 DDPG 的背景下使用它们，并在下一节中解释它们的实现。

3 算法

不可能直接将 Q 学习应用于连续动作空间，因为在连续空间中找到贪婪策略需要在每个时间步对 a_t 进行优化；这种优化速度太慢，对于大型、无约束的函数逼近器和非平凡的动作空间来说不切实际。相反，我们在这里使用了基于 DPG 算法的 actor-critic 方法 (Silver et al., 2014)。

DPG 算法维护一个参数化参与者函数 $\mu(s | \theta^\mu)$ ，它通过确定性地将状态映射到特定操作来指定当前策略。批评家 $Q(s, a)$ 是使用 Q 学习中的贝尔曼方程来学习的。通过将链式法则应用到相对于 actor 参数的起始分布 J 的预期回报来更新 actor：

$$\begin{aligned} \nabla_{\theta^\mu} J &\approx \mathbb{E}_{s_t \sim \rho^\beta} [\nabla_{\theta^\mu} Q(s, a | \theta^Q) |_{s=s_t, a=\mu(s_t | \theta^\mu)}] \\ &= \mathbb{E}_{s_t \sim \rho^\beta} [\nabla_a Q(s, a | \theta^Q) |_{s=s_t, a=\mu(s_t)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) |_{s=s_t}] \end{aligned} \quad (6)$$

银等人。(2014)省 得知这是 *policy gradient*，策略 μ 的梯度 性能²。

与 Q 学习一样，引入非线性函数逼近器意味着不再保证收敛。然而，为了在大状态空间上学习和泛化，这样的逼近器似乎是必不可少的。NFQCA (Hafner & Riedmiller, 2011) 使用与 DPG 相同的更新规则，但使用神经网络函数逼近器，使用批量学习来保持稳定性，这对于大型网络来说是棘手的。NFQCA 的小批量版本不会在每次更新时重置策略（需要扩展到大型网络），相当于我们在这里进行比较的原始 DPG。受 DQN 成功的启发，我们的贡献是对 DPG 进行修改，使其能够使用神经网络函数逼近器在大型状态和动作空间中在线学习。我们将我们的算法称为 Deep DPG (DDPG, 算法 1)。

²In practice, as in commonly done in policy gradient implementations, we ignored the discount in the state-visitation distribution ρ^β .

One challenge when using neural networks for reinforcement learning is that most optimization algorithms assume that the samples are independently and identically distributed. Obviously, when the samples are generated from exploring sequentially in an environment this assumption no longer holds. Additionally, to make efficient use of hardware optimizations, it is essential to learn in mini-batches, rather than online.

As in DQN, we used a replay buffer to address these issues. The replay buffer is a finite sized cache $\mathcal{R}$. Transitions were sampled from the environment according to the exploration policy and the tuple (s_t, a_t, r_t, s_{t+1}) was stored in the replay buffer. When the replay buffer was full the oldest samples were discarded. At each timestep the actor and critic are updated by sampling a minibatch uniformly from the buffer. Because DDPG is an off-policy algorithm, the replay buffer can be large, allowing the algorithm to benefit from learning across a set of uncorrelated transitions.

Directly implementing Q learning (equation 4) with neural networks proved to be unstable in many environments. Since the network $Q(s, a|\theta^Q)$ being updated is also used in calculating the target value (equation 5), the Q update is prone to divergence. Our solution is similar to the target network used in (Mnih et al., 2013) but modified for actor-critic and using “soft” target updates, rather than directly copying the weights. We create a copy of the actor and critic networks, $Q'(s, a|\theta^{Q'})$ and $\mu'(s|\theta^{\mu'})$ respectively, that are used for calculating the target values. The weights of these target networks are then updated by having them slowly track the learned networks: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ with $\tau \ll 1$. This means that the target values are constrained to change slowly, greatly improving the stability of learning. This simple change moves the relatively unstable problem of learning the action-value function closer to the case of supervised learning, a problem for which robust solutions exist. We found that having both a target μ' and Q' was required to have stable targets y_i in order to consistently train the critic without divergence. This may slow learning, since the target network delays the propagation of value estimations. However, in practice we found this was greatly outweighed by the stability of learning.

When learning from low dimensional feature vector observations, the different components of the observation may have different physical units (for example, positions versus velocities) and the ranges may vary across environments. This can make it difficult for the network to learn effectively and may make it difficult to find hyper-parameters which generalise across environments with different scales of state values.

One approach to this problem is to manually scale the features so they are in similar ranges across environments and units. We address this issue by adapting a recent technique from deep learning called *batch normalization* (Ioffe & Szegedy, 2015). This technique normalizes each dimension across the samples in a minibatch to have unit mean and variance. In addition, it maintains a running average of the mean and variance to use for normalization during testing (in our case, during exploration or evaluation). In deep networks, it is used to minimize covariance shift during training, by ensuring that each layer receives whitened input. In the low-dimensional case, we used batch normalization on the state input and all layers of the μ network and all layers of the Q network prior to the action input (details of the networks are given in the supplementary material). With batch normalization, we were able to learn effectively across many different tasks with differing types of units, without needing to manually ensure the units were within a set range.

A major challenge of learning in continuous action spaces is exploration. An advantage of off-policies algorithms such as DDPG is that we can treat the problem of exploration independently from the learning algorithm. We constructed an exploration policy μ' by adding noise sampled from a noise process $\mathcal{N}$ to our actor policy

$$\mu'(s_t) = \mu(s_t|\theta_t^\mu) + \mathcal{N} \quad (7)$$

$\mathcal{N}$ can be chosen to suit the environment. As detailed in the supplementary materials we used an Ornstein-Uhlenbeck process (Uhlenbeck & Ornstein, 1930) to generate temporally correlated exploration for exploration efficiency in physical control problems with inertia (similar use of auto-correlated noise was introduced in (Wawrzyński, 2015)).

4 RESULTS

We constructed simulated physical environments of varying levels of difficulty to test our algorithm. This included classic reinforcement learning environments such as cartpole, as well as difficult,

使用神经网络进行强化学习时的一个挑战是，大多数优化算法都假设样本是独立且同分布的。显然，当样本是通过在环境中顺序探索而生成时，该假设不再成立。此外，为了有效利用硬件优化，必须以小批量方式学习，而不是在线学习。

与 DQN 一样，我们使用重播缓冲区来解决这些问题。重播缓冲区是一个有限大小的缓存 $\mathcal{R}$ 。根据探索策略从环境中对转换进行采样，并将元组 (s_t, a_t, r_t, s_{t+1}) 存储在重播缓冲区中。当重播缓冲区已满时，最旧的样本将被丢弃。在每个时间步，演员和评论家都会通过从缓冲区中均匀采样小批量来进行更新。由于 DDPG 是一种离策略算法，因此重播缓冲区可能很大，从而使算法能够从一组不相关转换的学习中受益。

事实证明，用神经网络直接实现 Q 学习（方程 4）在许多环境中是不稳定的。由于正在更新的网络 $Q(s, a|\theta^Q)$ 也用于计算目标值（等式 5），因此 Q 更新容易出现发散。我们的解决方案与 (Mnih et al., 2013) 中使用的目标网络类似，但针对 actor-critic 进行了修改，并使用“软”目标更新，而不是直接复制权重。我们创建演员网络和评论家网络的副本，分别为 $Q'(s, a|\theta^{Q'})$ 和 $\mu'(s|\theta^{\mu'})$ ，用于计算目标值。然后，通过让这些目标网络缓慢跟踪学习网络来更新它们的权重： $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$ 和 $\tau \ll 1$ 。这意味着目标值被限制为缓慢变化，从而大大提高了学习的稳定性。这个简单的改变使学习动作价值函数的相对不稳定的问题更接近监督学习的情况，这个问题存在鲁棒的解决方案。我们发现，同时拥有目标 μ' 和 Q' 需要有稳定的目标 y_i ，以便一致地训练批评家而不产生分歧。这可能会减慢学习速度，因为目标网络延迟了价值估计的传播。然而，在实践中我们发现学习的稳定性远远超过了这一点。

当从低维特征向量观测值中学习时，观测值的不同组成部分可能具有不同的物理单位（例如，位置与速度），并且范围可能因环境而异。这可能使网络难以有效学习，并且可能难以找到能够在具有不同状态值规模的环境中泛化的超参数。

解决此问题的一种方法是手动缩放特征，使它们在不同环境和单元中处于相似的范围。我们通过采用一种名为 *batch normalization* (Ioffe & Szegedy, 2015) 的深度学习最新技术来解决这个问题。该技术对小批量中样本的每个维度进行标准化，以获得单位均值和方差。此外，它还维护均值和方差的运行平均值，以便在测试期间（在我们的例子中，在探索或评估期间）用于标准化。在深度网络中，它用于通过确保每一层接收白化输入来最小化训练期间的协方差偏移。在低维情况下，我们在动作输入之前对状态输入和 μ 网络的所有层以及 Q 网络的所有层使用批量归一化（补充材料中给出了网络的详细信息）。通过批量归一化，我们能够有效地学习具有不同类型单元的许多不同任务，而无需手动确保单元在设定范围内。

在连续行动空间中学习的一个主要挑战是探索。DDPG 等非策略算法的一个优点是我们可以独立于学习算法来处理探索问题。我们通过将从噪声过程 $\mathcal{N}$ 采样的噪声添加到我们的 Actor 策略中来构建探索策略 μ'

$$\mu'(s_t) = \mu(s_t|\theta_t^\mu) + \mathcal{N} \quad (7)$$

$\mathcal{N}$ 可以根据环境进行选择。正如补充材料中详细介绍的，我们使用 Ornstein-Uhlenbeck 过程 (Uhlenbeck & Ornstein, 1930) 生成时间相关探索，以提高惯性物理控制问题的探索效率（自相关噪声的类似使用在 (Wawrzyński, 2015) 中引入）。

4 个结果

我们构建了不同难度级别的模拟物理环境来测试我们的算法。这包括经典的强化学习环境，例如 cartpole，以及困难的、

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
 Initialize a random process $\mathcal{N}$ for action exploration
 Receive initial observation state s_1
 for t = 1, T **do**
 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
 Execute action a_t and observe reward r_t and observe new state s_{t+1}
 Store transition (s_t, a_t, r_t, s_{t+1}) in R
 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

 Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

 end for
end for

high dimensional tasks such as gripper, tasks involving contacts such as puck striking (*canada*) and locomotion tasks such as *cheetah* (Wawrzyński, 2009). In all domains but *cheetah* the actions were torques applied to the actuated joints. These environments were simulated using MuJoCo (Todorov et al., 2012). Figure 1 shows renderings of some of the environments used in the task (the supplementary contains details of the environments and you can view some of the learned policies at <https://goo.gl/J4PIAz>).

In all tasks, we ran experiments using both a low-dimensional state description (such as joint angles and positions) and high-dimensional renderings of the environment. As in DQN (Mnih et al., 2013; 2015), in order to make the problems approximately fully observable in the high dimensional environment we used action repeats. For each timestep of the agent, we step the simulation 3 timesteps, repeating the agent’s action and rendering each time. Thus the observation reported to the agent contains 9 feature maps (the RGB of each of the 3 renderings) which allows the agent to infer velocities using the differences between frames. The frames were downsampled to 64x64 pixels and the 8-bit RGB values were converted to floating point scaled to $[0, 1]$. See supplementary information for details of our network structure and hyperparameters.

We evaluated the policy periodically during training by testing it without exploration noise. Figure 2 shows the performance curve for a selection of environments. We also report results with components of our algorithm (i.e. the target network or batch normalization) removed. In order to perform well across all tasks, both of these additions are necessary. In particular, learning without a target network, as in the original work with DPG, is very poor in many environments.

Surprisingly, in some simpler tasks, learning policies from pixels is just as fast as learning using the low-dimensional state descriptor. This may be due to the action repeats making the problem simpler. It may also be that the convolutional layers provide an easily separable representation of state space, which is straightforward for the higher layers to learn on quickly.

Table 1 summarizes DDPG’s performance across all of the environments (results are averaged over 5 replicas). We normalized the scores using two baselines. The first baseline is the mean return from a naive policy which samples actions from a uniform distribution over the valid action space. The second baseline is iLQG (Todorov & Li, 2005), a planning based solver with full access to the

算法1 DDPG算法

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
 Initialize a random process $\mathcal{N}$ for action exploration
 Receive initial observation state s_1
 for t = 1, T **do**
 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
 Execute action a_t and observe reward r_t and observe new state s_{t+1}
 Store transition (s_t, a_t, r_t, s_{t+1}) in R
 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

 Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

 end for
end for

高维任务（例如抓手）、涉及接触的任务（例如冰球撞击 (*canada*) 和运动任务（例如 *cheetah* (Wawrzyński, 2009)）。在除 *cheetah* 之外的所有域中，动作都是施加到驱动关节的扭矩。这些环境是使用 MuJoCo 进行模拟的 (Todorov 等人, 2012)。图 1 显示了任务中使用的一些环境的渲染（补充包含环境的详细信息，您可以在 <https://goo.gl/J4PIAz> 查看一些学习的策略）。

在所有任务中，我们都使用低维状态描述（例如关节角度和位置）和环境的高维渲染进行实验。与 DQN 一样 (Mnih 等, 2013; 2015)，为了使问题在高维环境中近似完全可观察，我们使用了动作重复。对于代理的每个时间步长，我们将模拟分 3 个时间步长，每次重复代理的动作和渲染。因此，向代理报告的观察结果包含 9 个特征图（3 个渲染中每个渲染的 RGB），这允许代理使用帧之间的差异来推断速度。帧被下采样到 64x64 像素，8 位 RGB 值被转换为缩放到 [0, 1] 的浮点值。有关我们的网络结构和超参数的详细信息，请参阅补充信息。

我们在训练期间定期评估该策略，在没有探索噪声的情况下对其进行测试。图 2 显示了所选环境的性能曲线。我们还报告了删除了我们的算法组件（即目标网络或批量归一化）的结果。为了在所有任务中表现良好，这两项补充都是必要的。特别是，没有目标网络的学习（如 DPG 的原始工作）在许多环境中效果非常差。

令人惊讶的是，在一些更简单的任务中，从像素学习策略与使用低维状态描述符学习一样快。这可能是由于动作的重复使问题变得更简单。卷积层也可能提供了一种易于分离的状态空间表示，这对于高层来说很容易快速学习。

表 1 总结了 DDPG 在所有环境中的性能（结果是 5 个副本的平均值）。我们使用两个基线对分数进行标准化。第一个基线是朴素策略的平均回报，该策略从有效动作空间上的均匀分布中采样动作。第二个基线是 iLQG (Todorov & Li, 2005)，一个基于规划的求解器，可以完全访问

underlying physical model and its derivatives. We normalize scores so that the naive policy has a mean score of 0 and iLQG has a mean score of 1. DDPG is able to learn good policies on many of the tasks, and in many cases some of the replicas learn policies which are superior to those found by iLQG, even when learning directly from pixels.

It can be challenging to learn accurate value estimates. Q-learning, for example, is prone to over-estimating values (Hasselt, 2010). We examined DDPG’s estimates empirically by comparing the values estimated by Q after training with the true returns seen on test episodes. Figure 3 shows that in simple tasks DDPG estimates returns accurately without systematic biases. For harder tasks the Q estimates are worse, but DDPG is still able learn good policies.

To demonstrate the generality of our approach we also include Torcs, a racing game where the actions are acceleration, braking and steering. Torcs has previously been used as a testbed in other policy learning approaches (Koutník et al., 2014b). We used an identical network architecture and learning algorithm hyper-parameters to the physics tasks but altered the noise process for exploration because of the very different time scales involved. On both low-dimensional and from pixels, some replicas were able to learn reasonable policies that are able to complete a circuit around the track though other replicas failed to learn a sensible policy.

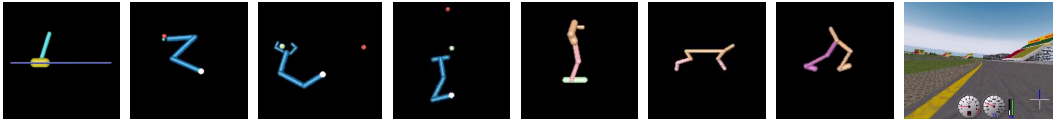


Figure 1: Example screenshots of a sample of environments we attempt to solve with DDPG. In order from the left: the cartpole swing-up task, a reaching task, a gasp and move task, a puck-hitting task, a monoped balancing task, two locomotion tasks and Torcs (driving simulator). We tackle all tasks using both low-dimensional feature vector and high-dimensional pixel inputs. Detailed descriptions of the environments are provided in the supplementary. Movies of some of the learned policies are available at <https://goo.gl/J4PIAz>.

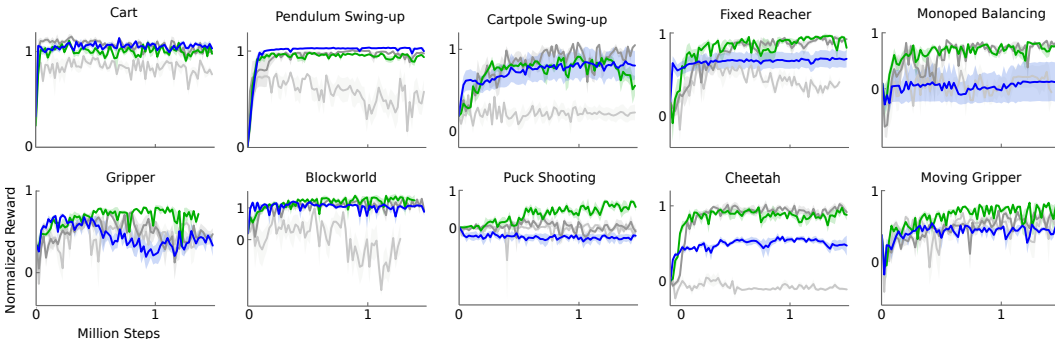


Figure 2: Performance curves for a selection of domains using variants of DPG: original DPG algorithm (minibatch NFQCA) with batch normalization (light grey), with target network (dark grey), with target networks and batch normalization (green), with target networks from pixel-only inputs (blue). Target networks are crucial.

5 RELATED WORK

The original DPG paper evaluated the algorithm with toy problems using tile-coding and linear function approximators. It demonstrated data efficiency advantages for off-policy DPG over both on- and off-policy stochastic actor critic. It also solved one more challenging task in which a multi-jointed octopus arm had to strike a target with any part of the limb. However, that paper did not demonstrate scaling the approach to large, high-dimensional observation spaces as we have here.

It has often been assumed that standard policy search methods such as those explored in the present work are simply too fragile to scale to difficult problems (Levine et al., 2015). Standard policy search

基础物理模型及其衍生模型。我们对分数进行标准化，使朴素策略的平均分数为 0，iLQG 的平均分数为 1。DDPG 能够在许多任务上学习良好的策略，并且在许多情况下，一些副本学习的策略优于 iLQG 发现的策略，即使直接从像素学习也是如此。

学习准确的价值估计可能具有挑战性。例如，Q 学习很容易高估值 (Hasselt, 2010)。我们通过将训练后 Q 估计的值与测试集上看到的真实回报进行比较，从经验上检查了 DDPG 的估计。图 3 显示，在简单任务中，DDPG 准确地估计回报，没有系统偏差。对于更困难的任务，Q 估计更差，但 DDPG 仍然能够学习好的策略。

为了展示我们方法的通用性，我们还加入了《Torcs》，这是一款赛车游戏，其中的动作包括加速、制动和转向。Torcs 之前曾被用作其他策略学习方法的测试平台 (Koutník 等人, 2014b)。我们对物理任务使用了相同的网络架构和学习算法超参数，但由于涉及的时间尺度非常不同，因此改变了探索的噪声过程。在低维和像素上，一些副本能够学习合理的策略，能够完成围绕赛道的一圈，尽管其他副本未能学习合理的策略。

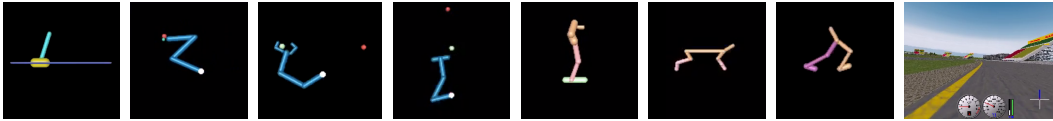


图 1: 我们尝试使用 DDPG 解决的环境示例的屏幕截图示例。从左起依次为：推杆摆动任务、伸手任务、喘息和移动任务、击球任务、单足平衡任务、两个运动任务和 Torcs（驾驶模拟器）。我们使用低维特征向量和高维像素输入来处理所有任务。补充材料中提供了环境的详细描述。一些学习政策的视频可在 <https://goo.gl/J4PIAz> 上找到。

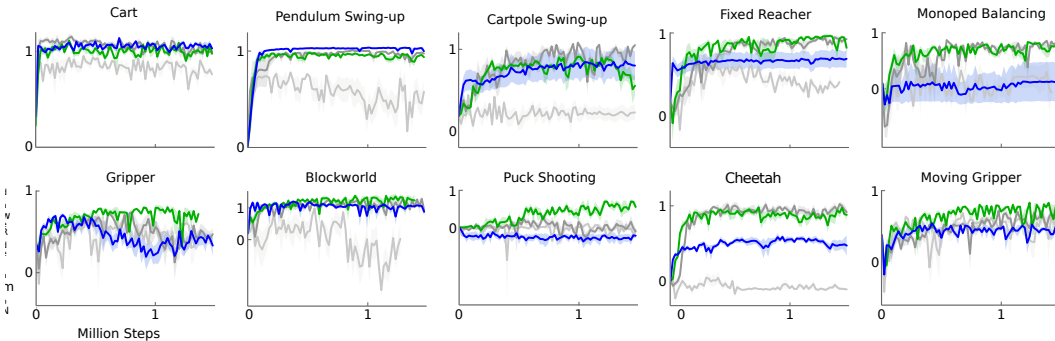


图 2: 使用 DPG 变体的选定域的性能曲线：具有批量归一化（浅灰色）的原始 DPG 算法（小批量 NFQCA）、具有目标网络（深灰色）、具有目标网络和批量归一化（绿色）、具有仅像素输入的目标网络（蓝色）。目标网络至关重要。

5 相关工作

最初的 DPG 论文使用平铺编码和线性函数逼近器评估了该算法的玩具问题。它证明了离策略 DPG 相对于在策略和离策略随机 Actor Critic 的数据效率优势。它还解决了一项更具挑战性的任务，即多关节章鱼臂必须用肢体的任何部分击中目标。然而，该论文并没有像我们这里那样展示将方法扩展到大型、高维观察空间的方法。

人们常常认为标准政策搜索方法（例如本工作中探索的方法）太脆弱，无法扩展到解决困难问题 (Levine 等人, 2015)。标准政策搜索

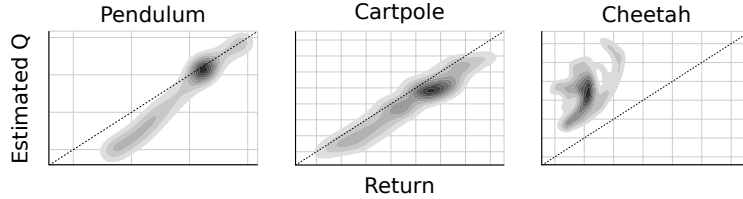


Figure 3: Density plot showing estimated Q values versus observed returns sampled from test episodes on 5 replicas. In simple domains such as pendulum and cartpole the Q values are quite accurate. In more complex tasks, the Q estimates are less accurate, but can still be used to learn competent policies. Dotted line indicates unity, units are arbitrary.

Table 1: Performance after training across all environments for at most 2.5 million steps. We report both the average and best observed (across 5 runs). All scores, except Torcs, are normalized so that a random agent receives 0 and a planning algorithm 1; for Torcs we present the raw reward score. We include results from the DDPG algorithm in the low-dimensional (*lowd*) version of the environment and high-dimensional (*pix*). For comparison we also include results from the original DPG algorithm with a replay buffer and batch normalization (*cntrl*).

environment	$R_{av,lowd}$	$R_{best,lowd}$	$R_{av,pix}$	$R_{best,pix}$	$R_{av,cntrl}$	$R_{best,cntrl}$
blockworld1	1.156	1.511	0.466	1.299	-0.080	1.260
blockworld3da	0.340	0.705	0.889	2.225	-0.139	0.658
canada	0.303	1.735	0.176	0.688	0.125	1.157
canada2d	0.400	0.978	-0.285	0.119	-0.045	0.701
cart	0.938	1.336	1.096	1.258	0.343	1.216
cartpole	0.844	1.115	0.482	1.138	0.244	0.755
cartpoleBalance	0.951	1.000	0.335	0.996	-0.468	0.528
cartpoleParallelDouble	0.549	0.900	0.188	0.323	0.197	0.572
cartpoleSerialDouble	0.272	0.719	0.195	0.642	0.143	0.701
cartpoleSerialTriple	0.736	0.946	0.412	0.427	0.583	0.942
cheetah	0.903	1.206	0.457	0.792	-0.008	0.425
fixedReacher	0.849	1.021	0.693	0.981	0.259	0.927
fixedReacherDouble	0.924	0.996	0.872	0.943	0.290	0.995
fixedReacherSingle	0.954	1.000	0.827	0.995	0.620	0.999
gripper	0.655	0.972	0.406	0.790	0.461	0.816
gripperRandom	0.618	0.937	0.082	0.791	0.557	0.808
hardCheetah	1.311	1.990	1.204	1.431	-0.031	1.411
hopper	0.676	0.936	0.112	0.924	0.078	0.917
hyq	0.416	0.722	0.234	0.672	0.198	0.618
movingGripper	0.474	0.936	0.480	0.644	0.416	0.805
pendulum	0.946	1.021	0.663	1.055	0.099	0.951
reacher	0.720	0.987	0.194	0.878	0.231	0.953
reacher3daFixedTarget	0.585	0.943	0.453	0.922	0.204	0.631
reacher3daRandomTarget	0.467	0.739	0.374	0.735	-0.046	0.158
reacherSingle	0.981	1.102	1.000	1.083	1.010	1.083
walker2d	0.705	1.573	0.944	1.476	0.393	1.397
torcs	-393.385	1840.036	-401.911	1876.284	-911.034	1961.600

is thought to be difficult because it deals simultaneously with complex environmental dynamics and a complex policy. Indeed, most past work with actor-critic and policy optimization approaches have had difficulty scaling up to more challenging problems (Deisenroth et al., 2013). Typically, this is due to instability in learning wherein progress on a problem is either destroyed by subsequent learning updates, or else learning is too slow to be practical.

Recent work with model-free policy search has demonstrated that it may not be as fragile as previously supposed. Wawrzyński (2009); Wawrzyński & Tanwani (2013) has trained stochastic policies

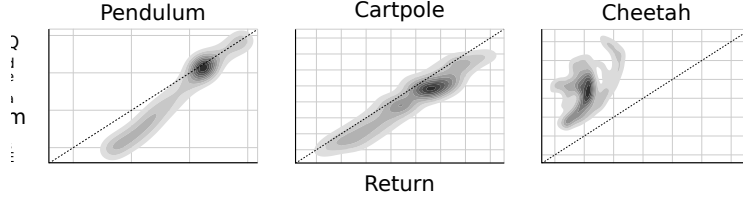


图 3: 密度图显示了估计的 Q 值与从 5 个副本的测试片段中采样的观察到的回报。在简单的领域（例如摆和推杆）中，Q 值非常准确。在更复杂的任务中，Q 估计不太准确，但仍然可以用来学习有效的策略。虚线表示统一，单位任意。

表 1: 在所有环境中训练最多 250 万步后的性能。我们报告了平均值和最佳观察值（5 次运行）。除 Torcs 之外的所有分数均已标准化，以便随机代理收到 0 分，规划算法收到 1 分；对于 Torcs，我们提供原始奖励分数。我们将 DDPG 算法的结果包含在环境的低维 (*lowd*) 版本和高维 (*pix*) 版本中。为了进行比较，我们还包括带有重播缓冲区和批量归一化 (*cntrl*) 的原始 DPG 算法的结果。

environment	$R_{av,lowd}$	$R_{best,lowd}$	$R_{av,pix}$	$R_{best,pix}$	$R_{av,cntrl}$	$R_{best,cntrl}$
blockworld1	1.156	1.511	0.466	1.299	-0.080	1.260
blockworld3da	0.340	0.705	0.889	2.225	-0.139	0.658
canada	0.303	1.735	0.176	0.688	0.125	1.157
canada2d	0.400	0.978	-0.285	0.119	-0.045	0.701
cart	0.938	1.336	1.096	1.258	0.343	1.216
cartpole	0.844	1.115	0.482	1.138	0.244	0.755
cartpoleBalance	0.951	1.000	0.335	0.996	-0.468	0.528
cartpoleParallelDouble	0.549	0.900	0.188	0.323	0.197	0.572
cartpoleSerialDouble	0.272	0.719	0.195	0.642	0.143	0.701
cartpoleSerialTriple	0.736	0.946	0.412	0.427	0.583	0.942
cheetah	0.903	1.206	0.457	0.792	-0.008	0.425
fixedReacher	0.849	1.021	0.693	0.981	0.259	0.927
fixedReacherDouble	0.924	0.996	0.872	0.943	0.290	0.995
fixedReacherSingle	0.954	1.000	0.827	0.995	0.620	0.999
gripper	0.655	0.972	0.406	0.790	0.461	0.816
gripperRandom	0.618	0.937	0.082	0.791	0.557	0.808
hardCheetah	1.311	1.990	1.204	1.431	-0.031	1.411
hopper	0.676	0.936	0.112	0.924	0.078	0.917
hyq	0.416	0.722	0.234	0.672	0.198	0.618
movingGripper	0.474	0.936	0.480	0.644	0.416	0.805
pendulum	0.946	1.021	0.663	1.055	0.099	0.951
reacher	0.720	0.987	0.194	0.878	0.231	0.953
reacher3daFixedTarget	0.585	0.943	0.453	0.922	0.204	0.631
reacher3daRandomTarget	0.467	0.739	0.374	0.735	-0.046	0.158
reacherSingle	0.981	1.102	1.000	1.083	1.010	1.083
walker2d	0.705	1.573	0.944	1.476	0.393	1.397
torcs	-393.385	1840.036	-401.911	1876.284	-911.034	1961.600

被认为是困难的，因为它同时涉及复杂的环境动态和复杂的政策。事实上，过去大多数使用行为批评家和政策优化方法的工作都难以扩展到更具挑战性的问题（Deisenroth 等，2013）。通常，这是由于学习的不稳定造成的，其中问题的进展要么被后续的学习更新所破坏，要么学习太慢而无法实用。

最近的无模型政策搜索工作表明，它可能并不像以前想象的那么脆弱。Wawrzyński (2009)；Wawrzyński & Tanwani (2013) 训练了随机策略

in an actor-critic framework with a replay buffer. Concurrent with our work, Balduzzi & Ghifary (2015) extended the DPG algorithm with a “deviator” network which explicitly learns $\partial Q/\partial a$. However, they only train on two low-dimensional domains. Heess et al. (2015) introduced SVG(0) which also uses a Q-critic but learns a stochastic policy. DPG can be considered the deterministic limit of SVG(0). The techniques we described here for scaling DPG are also applicable to stochastic policies by using the reparametrization trick (Heess et al., 2015; Schulman et al., 2015a).

Another approach, trust region policy optimization (TRPO) (Schulman et al., 2015b), directly constructs stochastic neural network policies without decomposing problems into optimal control and supervised phases. This method produces near monotonic improvements in return by making carefully chosen updates to the policy parameters, constraining updates to prevent the new policy from diverging too far from the existing policy. This approach does not require learning an action-value function, and (perhaps as a result) appears to be significantly less data efficient.

To combat the challenges of the actor-critic approach, recent work with guided policy search (GPS) algorithms (e.g., (Levine et al., 2015)) decomposes the problem into three phases that are relatively easy to solve: first, it uses full-state observations to create locally-linear approximations of the dynamics around one or more nominal trajectories, and then uses optimal control to find the locally-linear optimal policy along these trajectories; finally, it uses supervised learning to train a complex, non-linear policy (e.g. a deep neural network) to reproduce the state-to-action mapping of the optimized trajectories.

This approach has several benefits, including data efficiency, and has been applied successfully to a variety of real-world robotic manipulation tasks using vision. In these tasks GPS uses a similar convolutional policy network to ours with 2 notable exceptions: 1. it uses a spatial softmax to reduce the dimensionality of visual features into a single (x, y) coordinate for each feature map, and 2. the policy also receives direct low-dimensional state information about the configuration of the robot at the first fully connected layer in the network. Both likely increase the power and data efficiency of the algorithm and could easily be exploited within the DDPG framework.

PILCO (Deisenroth & Rasmussen, 2011) uses Gaussian processes to learn a non-parametric, probabilistic model of the dynamics. Using this learned model, PILCO calculates analytic policy gradients and achieves impressive data efficiency in a number of control problems. However, due to the high computational demand, PILCO is “impractical for high-dimensional problems” (Wahlström et al., 2015). It seems that deep function approximators are the most promising approach for scaling reinforcement learning to large, high-dimensional domains.

Wahlström et al. (2015) used a deep dynamical model network along with model predictive control to solve the pendulum swing-up task from pixel input. They trained a differentiable forward model and encoded the goal state into the learned latent space. They use model-predictive control over the learned model to find a policy for reaching the target. However, this approach is only applicable to domains with goal states that can be demonstrated to the algorithm.

Recently, evolutionary approaches have been used to learn competitive policies for Torcs from pixels using compressed weight parametrizations (Koutník et al., 2014a) or unsupervised learning (Koutník et al., 2014b) to reduce the dimensionality of the evolved weights. It is unclear how well these approaches generalize to other problems.

6 CONCLUSION

The work combines insights from recent advances in deep learning and reinforcement learning, resulting in an algorithm that robustly solves challenging problems across a variety of domains with continuous action spaces, even when using raw pixels for observations. As with most reinforcement learning algorithms, the use of non-linear function approximators nullifies any convergence guarantees; however, our experimental results demonstrate that stable learning without the need for any modifications between environments.

Interestingly, all of our experiments used substantially fewer steps of experience than was used by DQN learning to find solutions in the Atari domain. Nearly all of the problems we looked at were solved within 2.5 million steps of experience (and usually far fewer), a factor of 20 fewer steps than

在具有重放缓冲区的演员评论家框架中。与我们的工作同时，Balduzzi & Ghifary (2015) 使用显式学习 $\partial Q/\partial a$ 的“偏差”网络扩展了 DPG 算法。然而，他们只在两个低维域上进行训练。赫斯等人。(2015) 引入了 SVG(0)，它也使用 Q-critic，但学习随机策略。DPG 可以被认为是 SVG(0) 的确定性极限。我们在此描述的用于缩放 DPG 的技术也适用于通过使用重参数化技巧的随机策略 (Heess 等人, 2015; Schulman 等人, 2015a)。

另一种方法是信任域策略优化 (TRPO) (Schulman et al., 2015b)，直接构建随机神经网络策略，而不将问题分解为最优控制和监督阶段。该方法通过仔细选择策略参数的更新、限制更新以防止新策略与现有策略偏离太远，从而产生近乎单调的改进。这种方法不需要学习行动价值函数，并且 (也许因此) 数据效率似乎明显较低。

为了应对行动者批评家方法的挑战，最近使用引导策略搜索 (GPS) 算法 (例如，(Levine 等人, 2015)) 将问题分解为相对容易解决的三个阶段：首先，它使用全状态观察来创建围绕一个或多个标称轨迹的动态的局部线性近似，然后使用最优控制来沿着这些轨迹找到局部线性最优策略；最后，它使用监督学习来训练复杂的非线性策略 (例如深度神经网络)，以重现优化轨迹的状态到动作映射。

这种方法有很多好处，包括数据效率，并且已成功应用于各种使用视觉的现实世界机器人操作任务。在这些任务中，GPS 使用与我们类似的卷积策略网络，但有 2 个值得注意的例外：1. 它使用空间 softmax 将视觉特征的维度减少为每个特征图的单个 (x, y) 坐标，2. 该策略还接收有关网络中第一个全连接层的机器人配置的直接低维状态信息。两者都可能提高算法的能力和效率，并且可以在 DDPG 框架中轻松利用。

PILCO (Deisenroth & Rasmussen, 2011) 使用高斯过程来学习动力学的非参数概率模型。使用这种学习模型，PILCO 可以计算分析策略梯度，并在许多控制问题中实现令人印象深刻的数据效率。然而，由于高计算需求，PILCO “对于高维问题不切实际” (Wahlström et al., 2015)。深度函数逼近器似乎是将强化学习扩展到大型高维领域的最有前途的方法。

Wahlström 等人。(2015) 使用深度动态模型网络和模型预测控制来解决来自像素输入的摆锤摆动任务。他们训练了一个可微分的前向模型，并将目标状态编码到学习的潜在空间中。他们使用对学习模型的模型预测控制来找到实现目标的策略。然而，这种方法仅适用于具有可以向算法证明的目标状态的领域。

最近，进化方法已被用来从像素中学习 Torc 的竞争策略，使用压缩权重参数化 (Koutník 等人, 2014a) 或无监督学习 (Koutník 等人, 2014b) 来降低进化权重的维度。目前还不清楚这些方法对其他问题的推广效果如何。

6 结论

这项工作结合了深度学习和强化学习最新进展的见解，产生了一种算法，即使在使用原始像素进行观察时，也可以通过连续的动作空间稳健地解决跨多个领域的挑战性问题。与大多数强化学习算法一样，非线性函数逼近器的使用使任何收敛保证失效。然而，我们的实验结果表明，无需在环境之间进行任何修改即可稳定学习。

有趣的是，我们所有的实验使用的经验步骤比 DQN 学习在 Atari 领域寻找解决方案所使用的经验步骤要少得多。我们研究的几乎所有问题都在 250 万步 (通常要少得多) 的经验内得到了解决，比之前少了 20 步。

DQN requires for good Atari solutions. This suggests that, given more simulation time, DDPG may solve even more difficult problems than those considered here.

A few limitations to our approach remain. Most notably, as with most model-free reinforcement approaches, DDPG requires a large number of training episodes to find solutions. However, we believe that a robust model-free approach may be an important component of larger systems which may attack these limitations (Gläscher et al., 2010).

REFERENCES

- Balduzzi, David and Ghifary, Muhammad. Compatible value gradients for reinforcement learning of continuous deep policies. *arXiv preprint arXiv:1509.03005*, 2015.
- Deisenroth, Marc and Rasmussen, Carl E. Pilco: A model-based and data-efficient approach to policy search. In *Proceedings of the 28th International Conference on machine learning (ICML-11)*, pp. 465–472, 2011.
- Deisenroth, Marc Peter, Neumann, Gerhard, Peters, Jan, et al. A survey on policy search for robotics. *Foundations and Trends in Robotics*, 2(1-2):1–142, 2013.
- Gläscher, Jan, Daw, Nathaniel, Dayan, Peter, and O’Doherty, John P. States versus rewards: dissociable neural prediction error signals underlying model-based and model-free reinforcement learning. *Neuron*, 66(4):585–595, 2010.
- Glorot, Xavier, Bordes, Antoine, and Bengio, Yoshua. Deep sparse rectifier networks. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics. JMLR W&CP Volume*, volume 15, pp. 315–323, 2011.
- Hafner, Roland and Riedmiller, Martin. Reinforcement learning in feedback control. *Machine learning*, 84(1-2):137–169, 2011.
- Hasselt, Hado V. Double q-learning. In *Advances in Neural Information Processing Systems*, pp. 2613–2621, 2010.
- Heess, N., Hunt, J. J., Lillicrap, T. P., and Silver, D. Memory-based control with recurrent neural networks. *NIPS Deep Reinforcement Learning Workshop (arXiv:1512.04455)*, 2015.
- Heess, Nicolas, Wayne, Gregory, Silver, David, Lillicrap, Tim, Erez, Tom, and Tassa, Yuval. Learning continuous control policies by stochastic value gradients. In *Advances in Neural Information Processing Systems*, pp. 2926–2934, 2015.
- Ioffe, Sergey and Szegedy, Christian. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.
- Kingma, Diederik and Ba, Jimmy. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Koutník, Jan, Schmidhuber, Jürgen, and Gomez, Faustino. Evolving deep unsupervised convolutional networks for vision-based reinforcement learning. In *Proceedings of the 2014 conference on Genetic and evolutionary computation*, pp. 541–548. ACM, 2014a.
- Koutník, Jan, Schmidhuber, Jürgen, and Gomez, Faustino. Online evolution of deep convolutional network for vision-based reinforcement learning. In *From Animals to Animats 13*, pp. 260–269. Springer, 2014b.
- Krizhevsky, Alex, Sutskever, Ilya, and Hinton, Geoffrey E. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pp. 1097–1105, 2012.
- Levine, Sergey, Finn, Chelsea, Darrell, Trevor, and Abbeel, Pieter. End-to-end training of deep visuomotor policies. *arXiv preprint arXiv:1504.00702*, 2015.

DQN 需要良好的 Atari 解决方案。这表明，如果有更多的模拟时间，DDPG 可能会解决比此处考虑的问题更困难的问题。

我们的方法仍然存在一些局限性。最值得注意的是，与大多数无模型强化方法一样，DDPG 需要大量的训练集才能找到解决方案。然而，我们相信，稳健的无模型方法可能是大型系统的重要组成部分，可以突破这些限制（Gläscher 等人，2010）。

参考

大卫·巴尔杜齐和穆罕默德·吉法里。用于连续深度策略强化学习的兼容值梯度。

arXiv preprint arXiv:1509.03005, 2015. Deisenroth, Marc 和 Rasmussen, Carl E. Pilco: 基于模型和数据高效的策略搜索方法。

Proceedings of the 28th International Conference on machine learning (ICML- 11), 第 465–472 页, 2011 年. Deisenroth, Marc Peter, Neumann, Gerhard, Peters, Jan 等人。关于机器人政策搜索的调查。 *Foundations and Trends in Robotics*, 2(1-2): 1–142, 2013。

Gläscher, Jan, Daw, Nathaniel, Dayan, Peter, and O’ Doherty, John P. 状态与奖励：基于模型和无模型强化学习的可分离神经预测误差信号。 *Neuron*, 66(4): 585–595, 2010。

泽维尔·格洛洛特、安托万·博德斯和约书亚·本吉奥。深度稀疏整流网络。 *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics. JMLR W&CP Volume*, 第 15 卷, 第 315–323 页, 2011 年。

罗兰·哈夫纳和马丁·里德米勒。反馈控制中的强化学习。 *Machine learning*, 84(1-2): 137–169, 2011。

Hasselt, Hado V. 双 q 学习。 *Advances in Neural Information Processing Systems*, 第 2613–2621 页, 2010 年。

Heess, N., Hunt, J. J., Lillicrap, T. P 和 Silver, D. 使用循环神经网络进行基于记忆的控制。 *NIPS Deep Reinforcement Learning Workshop (arXiv:1512.04455)*, 2015。

Heess, Nicolas, Wayne, Gregory, Silver, David, Lillicrap, Tim, Erez, Tom 和 Tassa, Yuval. 通过随机值梯度学习连续控制策略。 *Advances in Neural Information Processing Systems*, 第 2926–2934 页, 2015 年。

约夫·谢尔盖和克里斯蒂安·塞格迪。批量归一化：通过减少内部协变量偏移来加速深度网络训练。 *arXiv preprint arXiv:1502.03167*, 2015。

金玛，迪德里克和巴，吉米。Adam：一种随机优化方法。 *arXiv preprint arXiv:1412.6980*, 2014 年。

Koutnik, Jan, Schmidhuber, Jürgen 和 Gomez, Faustino. 进化深度无监督卷积网络以实现基于视觉的强化学习。在 *Proceedings of the 2014 conference on Genetic and evolutionary computation*, 第 541–548 页。ACM, 2014a。

Koutnik, Jan, Schmidhuber, Jürgen 和 Gomez, Faustino. 用于基于视觉的强化学习的深度卷积网络的在线演化。在 *From Animals to Animats 13* 中, 第 260–269 页。施普林格, 2014b。

Krizhevsky, Alex, Sutskever, Ilya 和 Hinton, Geoffrey E. 深度卷积神经网络的 Imagenet 分类。 *Advances in neural information processing systems*, 第 1097–1105 页, 2012 年。

莱文、谢尔盖、芬恩、切尔西、达雷尔、特雷弗和阿比尔、彼得。深度视觉运动策略的端到端训练。 *arXiv preprint arXiv:1504.00702*, 2015。

- Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Graves, Alex, Antonoglou, Ioannis, Wierstra, Daan, and Riedmiller, Martin. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Rusu, Andrei A, Veness, Joel, Bellemare, Marc G, Graves, Alex, Riedmiller, Martin, Fidjeland, Andreas K, Ostrovski, Georg, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Prokhorov, Danil V, Wunsch, Donald C, et al. Adaptive critic designs. *Neural Networks, IEEE Transactions on*, 8(5):997–1007, 1997.
- Schulman, John, Heess, Nicolas, Weber, Theophane, and Abbeel, Pieter. Gradient estimation using stochastic computation graphs. In *Advances in Neural Information Processing Systems*, pp. 3510–3522, 2015a.
- Schulman, John, Levine, Sergey, Moritz, Philipp, Jordan, Michael I, and Abbeel, Pieter. Trust region policy optimization. *arXiv preprint arXiv:1502.05477*, 2015b.
- Silver, David, Lever, Guy, Heess, Nicolas, Degris, Thomas, Wierstra, Daan, and Riedmiller, Martin. Deterministic policy gradient algorithms. In *ICML*, 2014.
- Tassa, Yuval, Erez, Tom, and Todorov, Emanuel. Synthesis and stabilization of complex behaviors through online trajectory optimization. In *Intelligent Robots and Systems (IROS), 2012 IEEE/RSJ International Conference on*, pp. 4906–4913. IEEE, 2012.
- Todorov, Emanuel and Li, Weiwei. A generalized iterative lqg method for locally-optimal feedback control of constrained nonlinear stochastic systems. In *American Control Conference, 2005. Proceedings of the 2005*, pp. 300–306. IEEE, 2005.
- Todorov, Emanuel, Erez, Tom, and Tassa, Yuval. Mujoco: A physics engine for model-based control. In *Intelligent Robots and Systems (IROS), 2012 IEEE/RSJ International Conference on*, pp. 5026–5033. IEEE, 2012.
- Uhlenbeck, George E and Ornstein, Leonard S. On the theory of the brownian motion. *Physical review*, 36(5):823, 1930.
- Wahlström, Niklas, Schön, Thomas B, and Deisenroth, Marc Peter. From pixels to torques: Policy learning with deep dynamical models. *arXiv preprint arXiv:1502.02251*, 2015.
- Watkins, Christopher JCH and Dayan, Peter. Q-learning. *Machine learning*, 8(3-4):279–292, 1992.
- Wawrzyński, Paweł. Real-time reinforcement learning by sequential actor-critics and experience replay. *Neural Networks*, 22(10):1484–1497, 2009.
- Wawrzyński, Paweł. Control policy with autocorrelated noise in reinforcement learning for robotics. *International Journal of Machine Learning and Computing*, 5:91–95, 2015.
- Wawrzyński, Paweł and Tanwani, Ajay Kumar. Autonomous reinforcement learning with experience replay. *Neural Networks*, 41:156–167, 2013.

Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Graves, Alex, Antonoglou, Ioannis, Wierstra, Daan 和 Riedmiller, Martin。通过深度强化学习玩 Atari。 *arXiv preprint arXiv:1312.5602*, 2013 年。

Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Rusu, Andrei A, Veness, Joel, Bellemare, Marc G, Graves, Alex, Riedmiller, Martin, Fidjeland, Andreas K, Ostrovski, Georg 等。通过深度强化学习进行人类水平的控制。 *Nature*, 518 (7540) : 529–533, 2015。

普罗霍罗夫、丹尼尔 V、温施、唐纳德 C 等。自适应批评家设计。 *Neural Networks, IEEE Transactions on*, 8(5): 997–1007, 1997。

舒尔曼、约翰、赫斯、尼古拉斯、韦伯、塞奥芬和阿贝尔、彼得。使用随机计算图的梯度估计。 *Advances in Neural Information Processing Systems*, 第 3510–3522 页, 2015a。

舒尔曼、约翰、莱文、谢尔盖、莫里茨、菲利普、乔丹、迈克尔一世和阿比尔、彼得。信托区域政策优化。 *arXiv preprint arXiv:1502.05477*, 2015b。

Silver, David, Lever, Guy, Heess, Nicolas, Degris, Thomas, Wierstra, Daan 和 Riedmiller, Martin。确定性策略梯度算法。在 *ICML*, 2014 年。

塔萨、尤瓦尔、埃雷兹、汤姆和托多罗夫、伊曼纽尔。通过在线轨迹优化来合成和稳定复杂行为。在 *Intelligent Robots and Systems (IROS), 2012 IEEE/RSJ International Conference on*, 第 4906–4913 页。IEEE, 2012。

托多罗夫, 伊曼纽尔和李伟伟。用于约束非线性随机系统局部最优反馈控制的广义迭代 lqg 方法。 *American Control Conference, 2005. Proceedings of the 2005*, 第 300–306 页。IEEE, 2005。

托多罗夫、伊曼纽尔、埃雷兹、汤姆和塔萨、尤瓦尔。Mujoco: 用于基于模型的控制的物理引擎。 *Intelligent Robots and Systems (IROS), 2012 IEEE/RSJ International Conference on*, 第 5026–5033 页。IEEE, 2012 年。

乔治·E·乌伦贝克 (Uhlenbeck) 和伦纳德·S·奥恩斯坦 (Ornstein)。论布朗运动理论。 *Physical review*, 36(5):823, 1930。

Wahlstrom, Niklas, Schon, Thomas B 和 Deisenroth, Marc Peter。从像素到扭矩: 利用深度动态模型进行策略学习。 *arXiv preprint arXiv:1502.02251*, 2015 年。克里斯托弗·JCH 沃特金斯和彼得·达扬。Q-学习。 *Machine learning*, 8(3-4): 279–292, 1992。

Wawrzyński, Paweł。通过连续演员评论家和经验回放进行实时强化学习。 *Neural Networks*, 22(10): 1484–1497, 2009。

Wawrzyński, Paweł。机器人强化学习中自相关噪声的控制策略。 *International Journal of Machine Learning and Computing*, 5: 91–95, 2015。

Wawrzyński, Paweł 和 Tanwani, Ajay Kumar。具有经验回放的自主强化学习。 *Neural Networks*, 41: 156–167, 2013。

Supplementary Information: Continuous control with deep reinforcement learning

7 EXPERIMENT DETAILS

We used Adam (Kingma & Ba, 2014) for learning the neural network parameters with a learning rate of 10^{-4} and 10^{-3} for the actor and critic respectively. For Q we included L_2 weight decay of 10^{-2} and used a discount factor of $\gamma = 0.99$. For the soft target updates we used $\tau = 0.001$. The neural networks used the rectified non-linearity (Glorot et al., 2011) for all hidden layers. The final output layer of the actor was a tanh layer, to bound the actions. The low-dimensional networks had 2 hidden layers with 400 and 300 units respectively ($\approx 130,000$ parameters). Actions were not included until the 2nd hidden layer of Q . When learning from pixels we used 3 convolutional layers (no pooling) with 32 filters at each layer. This was followed by two fully connected layers with 200 units ($\approx 430,000$ parameters). The final layer weights and biases of both the actor and critic were initialized from a uniform distribution $[-3 \times 10^{-3}, 3 \times 10^{-3}]$ and $[3 \times 10^{-4}, 3 \times 10^{-4}]$ for the low dimensional and pixel cases respectively. This was to ensure the initial outputs for the policy and value estimates were near zero. The other layers were initialized from uniform distributions $[-\frac{1}{\sqrt{f}}, \frac{1}{\sqrt{f}}]$ where f is the fan-in of the layer. The actions were not included until the fully-connected layers. We trained with minibatch sizes of 64 for the low dimensional problems and 16 on pixels. We used a replay buffer size of 10^6 .

For the exploration noise process we used temporally correlated noise in order to explore well in physical environments that have momentum. We used an Ornstein-Uhlenbeck process (Uhlenbeck & Ornstein, 1930) with $\theta = 0.15$ and $\sigma = 0.2$. The Ornstein-Uhlenbeck process models the velocity of a Brownian particle with friction, which results in temporally correlated values centered around 0.

8 PLANNING ALGORITHM

Our planner is implemented as a model-predictive controller (Tassa et al., 2012): at every time step we run a single iteration of trajectory optimization (using iLQG, (Todorov & Li, 2005)), starting from the true state of the system. Every single trajectory optimization is planned over a horizon between 250ms and 600ms, and this planning horizon recedes as the simulation of the world unfolds, as is the case in model-predictive control.

The iLQG iteration begins with an initial rollout of the previous policy, which determines the nominal trajectory. We use repeated samples of simulated dynamics to approximate a linear expansion of the dynamics around every step of the trajectory, as well as a quadratic expansion of the cost function. We use this sequence of locally-linear-quadratic models to integrate the value function backwards in time along the nominal trajectory. This *back-pass* results in a putative modification to the action sequence that will decrease the total cost. We perform a derivative-free line-search over this direction in the space of action sequences by integrating the dynamics forward (the *forward-pass*), and choose the best trajectory. We store this action sequence in order to warm-start the next iLQG iteration, and execute the first action in the simulator. This results in a new state, which is used as the initial state in the next iteration of trajectory optimization.

9 ENVIRONMENT DETAILS

9.1 TORCS ENVIRONMENT

For the Torcs environment we used a reward function which provides a positive reward at each step for the velocity of the car projected along the track direction and a penalty of -1 for collisions. Episodes were terminated if progress was not made along the track after 500 frames.

补充信息：深度强化学习的连续控制

7 实验细节

我们使用 Adam (Kingma & Ba, 2014) 来学习神经网络参数，演员和评论家的学习率分别为 10^{-4} 和 10^{-3} 。对于 Q ，我们加入了 10^{-2} 的 L_2 权重衰减，并使用了 $\gamma = 0.99$ 的折扣因子。对于软目标更新，我们使用 $\tau = 0.001$ 。神经网络对所有隐藏层使用修正的非线性 (Glorot 等人, 2011)。Actor 的最终输出层是 $\tanh$ 层，用于限制动作。低维网络有 2 个隐藏层，分别有 400 和 300 个单元 ($\approx 130,000$ 个参数)。直到 Q 的第二个隐藏层才包含操作。当从像素学习时，我们使用 3 个卷积层 (无池化)，每层有 32 个滤波器。接下来是两个具有 200 个单元的全连接层 ($\approx 430,000$ 个参数)。演员和评论家的最终层权重和偏差分别针对低维和像素情况从均匀分布 $[-3 \times 10^{-3}, 3 \times 10^{-3}]$ 和 $[3 \times 10^{-4}, 3 \times 10^{-4}]$ 初始化。这是为了确保政策和价值估计的初始输出接近于零。其他层从均匀分布 $[-\frac{1}{\sqrt{f}}, \frac{1}{\sqrt{f}}]$ 初始化，其中 f 是该层的扇入。直到完全连接的层才包含这些操作。我们针对低维问题使用 64 的小批量大小进行训练，针对像素问题使用 16 的小批量大小进行训练。我们使用的重播缓冲区大小为 10^6 。

对于探索噪声过程，我们使用时间相关噪声，以便在具有动量的物理环境中进行良好探索。我们使用了 Ornstein-Uhlenbeck 过程 (Uhlenbeck & Ornstein, 1930)，其中 $\theta = 0.15$ 和 $\sigma = 0.2$ 。Ornstein-Uhlenbeck 过程对带有摩擦的布朗粒子的速度进行建模，从而产生以 0 为中心的时间相关值。

8 规划算法

我们的规划器被实现为模型预测控制器 (Tassa et al., 2012)：在每个时间步，我们从系统的真实状态开始运行轨迹优化的单次迭代 (使用 iLQG, (Todorov & Li, 2005))。每个轨迹优化都是在 250 毫秒到 600 毫秒之间的范围内规划的，并且随着世界模拟的展开，这个规划范围会后退，就像模型预测控制的情况一样。

iLQG 迭代从先前策略的初始部署开始，这决定了标称轨迹。我们使用模拟动力学的重复样本来近似轨迹每一步的动力学线性展开，以及成本函数的二次展开。我们使用这一系列局部线性二次模型来沿着标称轨迹在时间上向后积分价值函数。此 *back-pass* 会导致对操作序列的假定修改，从而降低总成本。我们通过向前积分动态 (*forward-pass*)，在动作序列空间的这个方向上执行无导数线搜索，并选择最佳轨迹。我们存储这个动作序列，以便热启动下一个 iLQG 迭代，并在模拟器中执行第一个动作。这会产生一个新状态，该状态将用作轨迹优化的下一次迭代中的初始状态。

9 环境细节

9.1 托克斯环境

对于 Torcs 环境，我们使用了奖励函数，该函数在每一步为汽车沿赛道方向投射的速度提供正奖励，并为碰撞提供 -1 的惩罚。如果 500 帧后没有沿着轨道取得进展，则剧集将终止。

9.2 MUJoCo ENVIRONMENTS

For physical control tasks we used reward functions which provide feedback at every step. In all tasks, the reward contained a small action cost. For all tasks that have a static goal state (e.g. pendulum swingup and reaching) we provide a smoothly varying reward based on distance to a goal state, and in some cases an additional positive reward when within a small radius of the target state. For grasping and manipulation tasks we used a reward with a term which encourages movement towards the payload and a second component which encourages moving the payload to the target. In locomotion tasks we reward forward action and penalize hard impacts to encourage smooth rather than hopping gaits (Schulman et al., 2015b). In addition, we used a negative reward and early termination for falls which were determined by simple thresholds on the height and torso angle (in the case of walker2d).

Table 2 states the dimensionality of the problems and below is a summary of all the physics environments.

task name	dim(s)	dim(a)	dim(o)
blockworld1	18	5	43
blockworld3da	31	9	102
canada	22	7	62
canada2d	14	3	29
cart	2	1	3
cartpole	4	1	14
cartpoleBalance	4	1	14
cartpoleParallelDouble	6	1	16
cartpoleParallelTriple	8	1	23
cartpoleSerialDouble	6	1	14
cartpoleSerialTriple	8	1	23
cheetah	18	6	17
fixedReacher	10	3	23
fixedReacherDouble	8	2	18
fixedReacherSingle	6	1	13
gripper	18	5	43
gripperRandom	18	5	43
hardCheetah	18	6	17
hardCheetahNice	18	6	17
hopper	14	4	14
hyq	37	12	37
hyqKick	37	12	37
movingGripper	22	7	49
movingGripperRandom	22	7	49
pendulum	2	1	3
reacher	10	3	23
reacher3daFixedTarget	20	7	61
reacher3daRandomTarget	20	7	61
reacherDouble	6	1	13
reacherObstacle	18	5	38
reacherSingle	6	1	13
walker2d	18	6	41

Table 2: Dimensionality of the MuJoCo tasks: the dimensionality of the underlying physics model $\dim(s)$, number of action dimensions $\dim(a)$ and observation dimensions $\dim(o)$.

task name	Brief Description
blockworld1	Agent is required to use an arm with gripper constrained to the 2D plane to grab a falling block and lift it against gravity to a fixed target position.

9.2 MUJOCO 环境

对于物理控制任务，我们使用了奖励函数，它在每一步提供反馈。在所有任务中，奖励都包含少量的行动成本。对于具有静态目标状态的所有任务（例如钟摆摆动和到达），我们根据到目标状态的距离提供平滑变化的奖励，并且在某些情况下，在目标状态的小半径内时提供额外的正奖励。对于抓取和操作任务，我们使用了奖励，其中一个术语鼓励向有效负载移动，第二个组件鼓励将有效负载移动到目标。在运动任务中，我们奖励向前的动作并惩罚猛烈的撞击，以鼓励平稳而不是跳跃的步态（Schulman et al., 2015b）。此外，我们对跌倒使用了负奖励和提前终止，这是由高度和躯干角度的简单阈值决定的（在 walker2d 的情况下）。

表 2 列出了问题的维度，下面是所有物理环境的总结。

task name	dim(s)	dim(a)	dim(o)
blockworld1	18	5	43
blockworld3da	31	9	102
canada	22	7	62
canada2d	14	3	29
cart	2	1	3
cartpole	4	1	14
cartpoleBalance	4	1	14
cartpoleParallelDouble	6	1	16
cartpoleParallelTriple	8	1	23
cartpoleSerialDouble	6	1	14
cartpoleSerialTriple	8	1	23
cheetah	18	6	17
fixedReacher	10	3	23
fixedReacherDouble	8	2	18
fixedReacherSingle	6	1	13
gripper	18	5	43
gripperRandom	18	5	43
hardCheetah	18	6	17
hardCheetahNice	18	6	17
hopper	14	4	14
hyq	37	12	37
hyqKick	37	12	37
movingGripper	22	7	49
movingGripperRandom	22	7	49
pendulum	2	1	3
reacher	10	3	23
reacher3daFixedTarget	20	7	61
reacher3daRandomTarget	20	7	61
reacherDouble	6	1	13
reacherObstacle	18	5	38
reacherSingle	6	1	13
walker2d	18	6	41

表 2: MuJoCo 任务的维度：底层物理模型的维度 $\dim(s)$ 、动作维度数量 $\dim(a)$ 和观察维度 $\dim(o)$ 。

task name	Brief Description
blockworld1	Agent is required to use an arm with gripper constrained to the 2D plane to grab a falling block and lift it against gravity to a fixed target position.

blockworld3da	Agent is required to use a human-like arm with 7-DOF and a simple gripper to grab a block and lift it against gravity to a fixed target position.
canada	Agent is required to use a 7-DOF arm with hockey-stick like appendage to hit a ball to a target.
canada2d	Agent is required to use an arm with hockey-stick like appendage to hit a ball initialized to a random start location to a random target location.
cart	Agent must move a simple mass to rest at 0. The mass begins each trial in random positions and with random velocities.
cartpole	The classic cart-pole swing-up task. Agent must balance a pole attached to a cart by applying forces to the cart alone. The pole starts each episode hanging upside-down.
cartpoleBalance	The classic cart-pole balance task. Agent must balance a pole attached to a cart by applying forces to the cart alone. The pole starts in the upright positions at the beginning of each episode.
cartpoleParallelDouble	Variant on the classic cart-pole. Two poles, both attached to the cart, should be kept upright as much as possible.
cartpoleSerialDouble	Variant on the classic cart-pole. Two poles, one attached to the cart and the second attached to the end of the first, should be kept upright as much as possible.
cartpoleSerialTriple	Variant on the classic cart-pole. Three poles, one attached to the cart, the second attached to the end of the first, and the third attached to the end of the second, should be kept upright as much as possible.
cheetah	The agent should move forward as quickly as possible with a cheetah-like body that is constrained to the plane. This environment is based very closely on the one introduced by Wawrzyński (2009); Wawrzyński & Tanwani (2013).
fixedReacher	Agent is required to move a 3-DOF arm to a fixed target position.
fixedReacherDouble	Agent is required to move a 2-DOF arm to a fixed target position.
fixedReacherSingle	Agent is required to move a simple 1-DOF arm to a fixed target position.
gripper	Agent must use an arm with gripper appendage to grasp an object and maneuver the object to a fixed target.
gripperRandom	The same task as <code>gripper</code> except that the arm object and target position are initialized in random locations.
hardCheetah	The agent should move forward as quickly as possible with a cheetah-like body that is constrained to the plane. This environment is based very closely on the one introduced by Wawrzyński (2009); Wawrzyński & Tanwani (2013), but has been made much more difficult by removing the stabilizing joint stiffness from the model.
hopper	Agent must balance a multiple degree of freedom monopod to keep it from falling.
hyq	Agent is required to keep a quadroped model based on the hyq robot from falling.

blockworld3da	Agent is required to use a human-like arm with 7-DOF and a simple gripper to grab a block and lift it against gravity to a fixed target position.
canada	Agent is required to use a 7-DOF arm with hockey-stick like appendage to hit a ball to a target.
canada2d	Agent is required to use an arm with hockey-stick like appendage to hit a ball initialized to a random start location to a random target location.
cart	Agent must move a simple mass to rest at 0. The mass begins each trial in random positions and with random velocities.
cartpole	The classic cart-pole swing-up task. Agent must balance a pole attached to a cart by applying forces to the cart alone. The pole starts each episode hanging upside-down.
cartpoleBalance	The classic cart-pole balance task. Agent must balance a pole attached to a cart by applying forces to the cart alone. The pole starts in the upright positions at the beginning of each episode.
cartpoleParallelDouble	Variant on the classic cart-pole. Two poles, both attached to the cart, should be kept upright as much as possible.
cartpoleSerialDouble	Variant on the classic cart-pole. Two poles, one attached to the cart and the second attached to the end of the first, should be kept upright as much as possible.
cartpoleSerialTriple	Variant on the classic cart-pole. Three poles, one attached to the cart, the second attached to the end of the first, and the third attached to the end of the second, should be kept upright as much as possible.
cheetah	The agent should move forward as quickly as possible with a cheetah-like body that is constrained to the plane. This environment is based very closely on the one introduced by Wawrzyński (2009); Wawrzyński & Tanwani (2013).
fixedReacher	Agent is required to move a 3-DOF arm to a fixed target position.
fixedReacherDouble	Agent is required to move a 2-DOF arm to a fixed target position.
fixedReacherSingle	Agent is required to move a simple 1-DOF arm to a fixed target position.
gripper	Agent must use an arm with gripper appendage to grasp an object and maneuver the object to a fixed target.
gripperRandom	The same task as <code>gripper</code> except that the arm object and target position are initialized in random locations.
hardCheetah	The agent should move forward as quickly as possible with a cheetah-like body that is constrained to the plane. This environment is based very closely on the one introduced by Wawrzyński (2009); Wawrzyński & Tanwani (2013), but has been made much more difficult by removing the stabilizing joint stiffness from the model.
hopper	Agent must balance a multiple degree of freedom monopod to keep it from falling.
hyq	Agent is required to keep a quadroped model based on the hyq robot from falling.

movingGripper	Agent must use an arm with gripper attached to a moveable platform to grasp an object and move it to a fixed target.
movingGripperRandom	The same as the movingGripper environment except that the object position, target position, and arm state are initialized randomly.
pendulum	The classic pendulum swing-up problem. The pendulum should be brought to the upright position and balanced. Torque limits prevent the agent from swinging the pendulum up directly.
reacher3daFixedTarget	Agent is required to move a 7-DOF human-like arm to a fixed target position.
reacher3daRandomTarget	Agent is required to move a 7-DOF human-like arm from random starting locations to random target positions.
reacher	Agent is required to move a 3-DOF arm from random starting locations to random target positions.
reacherSingle	Agent is required to move a simple 1-DOF arm from random starting locations to random target positions.
reacherObstacle	Agent is required to move a 5-DOF arm around an obstacle to a randomized target position.
walker2d	Agent should move forward as quickly as possible with a bipedal walker constrained to the plane without falling down or pitching the torso too far forward or backward.

movingGripper	Agent must use an arm with gripper attached to a moveable platform to grasp an object and move it to a fixed target.
movingGripperRandom	The same as the movingGripper environment except that the object position, target position, and arm state are initialized randomly.
pendulum	The classic pendulum swing-up problem. The pendulum should be brought to the upright position and balanced. Torque limits prevent the agent from swinging the pendulum up directly.
reacher3daFixedTarget	Agent is required to move a 7-DOF human-like arm to a fixed target position.
reacher3daRandomTarget	Agent is required to move a 7-DOF human-like arm from random starting locations to random target positions.
reacher	Agent is required to move a 3-DOF arm from random starting locations to random target positions.
reacherSingle	Agent is required to move a simple 1-DOF arm from random starting locations to random target positions.
reacherObstacle	Agent is required to move a 5-DOF arm around an obstacle to a randomized target position.
walker2d	Agent should move forward as quickly as possible with a bipedal walker constrained to the plane without falling down or pitching the torso too far forward or backward.